

```

timescale 1ns / 1ps
`include "parameters.vh"

module csr_ssm_tb;

// Testbench parameters
parameter OPCODE_WIDTH = 8;
parameter TRAP_ADDRESS = 32'h100;

// Clock and reset
reg i_clk, i_rst_n;

// Interrupts and exceptions
reg i_external_interrupt, i_software_interrupt;
reg i_is_inst_illegal, i_is_ecall, i_is_ebreak, i_is_mret;

// Inputs
reg [OPCODE_WIDTH-1:0] i_opcode;
reg [31:0] i_y;
reg [2:0] i_funct3;
reg [11:0] i_csr_index;
reg [31:0] i_imm;
reg [31:0] i_rs1;
reg [31:0] i_pc;
reg writeback_change_pc;

// Outputs
wire [31:0] o_csr_out;
wire [31:0] o_return_address;
wire [31:0] o_trap_address;
wire o_go_to_trap_q;
wire o_return_from_trap_q;

// Instantiate the CSR module
csr_ssm #(OPCODE_WIDTH, TRAP_ADDRESS) uut (
    .i_clk(i_clk),
    .i_rst_n(i_rst_n),
    .i_external_interrupt(i_external_interrupt),
    .i_software_interrupt(i_software_interrupt),
    .i_is_inst_illegal(i_is_inst_illegal),
    .i_is_ecall(i_is_ecall),
    .i_is_ebreak(i_is_ebreak),
    .i_is_mret(i_is_mret),
    .i_opcode(i_opcode),
    .i_y(i_y),
    .i_funct3(i_funct3),

```

```

.i_csr_index(i_csr_index),
.i_imm(i_imm),
.i_rs1(i_rs1),
.o_csr_out(o_csr_out),
.i_pc(i_pc),
.writeback_change_pc(writeback_change_pc),
.o_return_address(o_return_address),
.o_trap_address(o_trap_address),
.o_go_to_trap_q(o_go_to_trap_q),
.o_return_from_trap_q(o_return_from_trap_q)
);

// Clock Generation
always #5 i_clk = ~i_clk; // 10 ns clock period

// Test Procedure
initial begin
    $dumpfile("csr_ssm_tb.vcd");
    $dumpvars(0, csr_ssm_tb);

    // Initialize inputs
    i_clk = 0;
    i_rst_n = 0;
    i_external_interrupt = 0;
    i_software_interrupt = 0;
    i_is_inst_illegal = 0;
    i_is_ecall = 0;
    i_is_ebreak = 0;
    i_is_mret = 0;
    i_opcode = 0;
    i_y = 0;
    i_funct3 = 0;
    i_csr_index = 0;
    i_imm = 0;
    i_rs1 = 0;
    i_pc = 0;
    writeback_change_pc = 0;

    // Reset sequence
    #10 i_rst_n = 1;
    #10;

    // Test 1: Write to MSTATUS register (CSRRS)
    i_opcode = `CSR;
    i_funct3 = 3'b010; // CSRRS
    i_csr_index = 12'h300; // MSTATUS

```

```

i_rs1 = 32'h00000001;
#10;

// Test 2: Read from MSTATUS register
i_funct3 = 3'b000; // CSR read
i_csr_index = 12'h300; // MSTATUS
#10;

// Test 3: Writing to MEPC register
i_funct3 = 3'b001; // CSRRW
i_csr_index = 12'h341; // MEPC
i_rs1 = 32'h12345678;
#10;

// Test 4: Checking if MEPC is read correctly
i_funct3 = 3'b000; // CSR read
i_csr_index = 12'h341;
#10;

// Test 5: Triggering an ECALL (Should set MEPC and go to TRAP)
i_is_ecall = 1;
i_pc = 32'h400000;
#10;
i_is_ecall = 0;
#10;

// Test 6: Checking if trap occurred
if (o_go_to_trap_q)
    $display("Trap triggered successfully! Trap Address: %h", o_trap_address);
else
    $display("Trap did not trigger!");

// Test 7: Executing MRET (Should return from trap)
i_is_mret = 1;
#10;
i_is_mret = 0;
#10;

// Checking return address
if (o_return_from_trap_q)
    $display("Returned from trap successfully! Return Address: %h",
o_return_address);
else
    $display("Return from trap failed!");

// End Simulation

```

```
#20;  
$finish;  
end  
  
endmodule
```